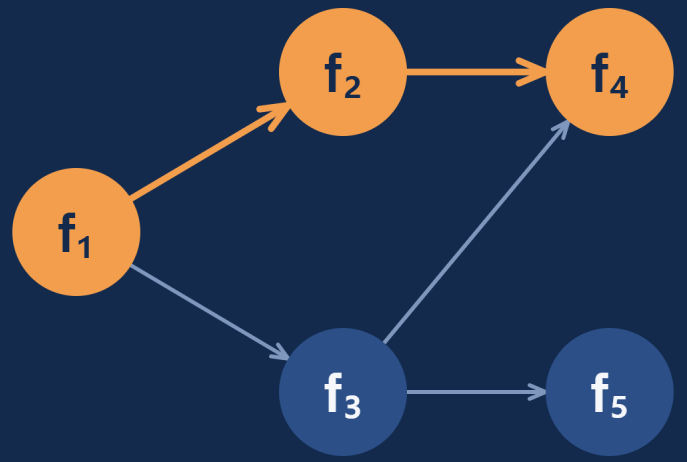
CLOUD COMPUTING · PROJECT PHASE 1 · 1ST EVALUATION

Predictive, Cold-Start-Aware Workflow Scheduling for Serverless Computing

AP-CWS sizes warm-container pools and places functions using each function's position in the workflow DAG.



Orange: the critical path, kept warm first. Blue: functions with slack.

Nikhil Sharma
24135083

Raghav Arora
24135097

Jashanpreet Singh
24134015

IIT (BHU) Varanasi · Instructor: [name] · October 2026 · Base paper: Pan et al., IEEE Transactions on Sustainable Computing, 2024

1 Problem Formulation

Background and Challenges

Cold-start latency

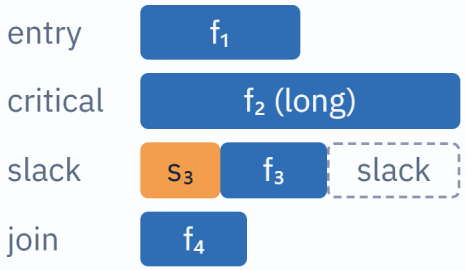
- No warm container: start one first
- Many warm: idle memory is paid for
- Few warm: frequent cold starts

Workflow resource allocation

- Apps are DAGs of dependent functions
- Cold starts add up along a path
- Per-function choices ignore the DAG

Why a Function's Position Matters

Fan-out $f_1 \rightarrow \{f_2, f_3\} \rightarrow f_4$ on a timeline



A cold start on f_3 (orange: startup delay) is hidden by slack. The same delay on f_2 delays the whole workflow.

Optimisation Objectives

- Minimise p95 workflow time
- Minimise cold-start rate
- Minimise idle warm memory
- Minimise resource cost
- Meet the latency SLO

p95: 95th-percentile completion time

Existing Approach: SSC (Pan et al., 2024)

1. Demand trend → Prewarm count
2. DAG → Critical path → CPU, memory

The two stages share nothing: prewarming ignores the critical path, and the critical path ignores likely cold starts.

Research Problem

How should a serverless platform size warm pools and place functions **together**, so that cold starts are prevented on a workflow's critical path and tolerated where it has slack?

Given	Workflow DAGs · m_i, t_i, s_i per function · invocation history · nodes
Decide	Warm pool W_i every minute · node x_{ij} for each ready function
Subject to	Node memory · DAG dependencies · SLO (service-level objective)

2 Literature Review and Research Gap

Recent Related Work (2024–2026)

Work	Approach	What it leaves open
Pan et al. 2024, SSC (base)	Trend prewarming; critical-path resources	Two stages; no shared information
Hanaforoosh et al. 2025	Granularity trees share dependency layers	Single functions; no DAG, no forecast
Chen et al. 2025, CASR	Keep-alive as caching, tuned by RL	No DAG; no node choice
Zhou et al. 2025	RL placement on heterogeneous nodes	No cold-start risk or workflow view
Nguyen et al. 2025, MPC	Forecast, prewarm, briefly delay requests	Independent functions; no placement
Zolfaghari et al. 2025	Edge workflow placement (genetic algorithm)	Placement not driven by cold-start risk
Alquraan et al. 2026, DROPS	Pre-warmed pool sizing to meet an SLO	Platform pools; no workflow criticality

G1 · Pools follow the demand trend, not a cold-start target

G2 · Likely cold starts never change what is critical

G3 · No node is chosen by where warm containers already are

From Separate Stages to One Decision

SSC and earlier work: separate stages

Demand trend → Prewarm pool

DAG → Critical path → Resources

AP-CWS: one joint decision

Forecast $\hat{\lambda}_i$ | Cold-start risk p_i

Criticality K_i | Node state

↓

Pool sizes W_i and node choice j^*

Research Gap

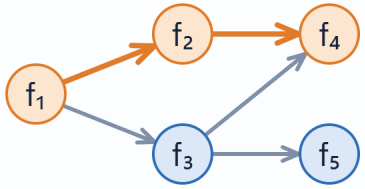
In the work we reviewed, predicted cold-start risk and workflow criticality are not combined in one decision about warm-pool size and function placement.

RL: reinforcement learning · MPC: model predictive control · full references on slide 7

3 Mathematical Modelling of the Proposed Solution

1. Workflow Model

$G = (V, E)$
$V = \{f_1, \ldots, f_n\}$: functions
E : dependency edges
$f_h \to f_i$: f_i starts after f_h



Function and Node Attributes

$f_i = (m_i,\ t_i,\ s_i)$
m_i : memory (GB)
t_i : warm execution time
s_i : startup (cold-start) delay
$j = (M_j,\ \pi_j,\ v_j)$: memory, price, speed

Decision Variables

$W_i(t) \in \mathbb{Z}_{\geq 0}$
warm containers of f_i, every Δ
$x_{ij} \in \{0, 1\}$
1 if f_i runs on node j

2. Demand Forecast (Holt)

$$\ell_i \leftarrow \theta\,\lambda_i + (1-\theta)(\ell_i + b_i)$$
$$b_i \leftarrow \varphi\,\Delta\ell_i + (1-\varphi)\,b_i$$
$$\hat{\lambda}_i(t+1) = \ell_i + b_i$$

Level + trend, like SSC's gradient, but smoothed.

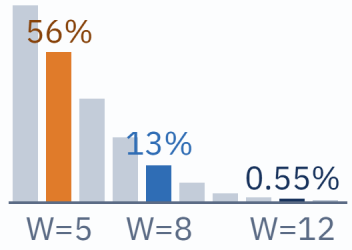
3. Cold-Start Probability and Pool Size · G1

$\rho_i = \hat{\lambda}_i\, t_i \quad$ (Little's law)

$$p_i(W) = \Pr[N_i \geq W] = 1 - \sum_{k=0}^{W-1} e^{-\rho_i} \frac{\rho_i^{\,k}}{k!}$$

$$W_i^* = \min\{\, W : p_i(W) \leq \varepsilon_i \,\}$$

ρ = 5



4. Workflow Criticality · G2

$d_i = t_i + p_i\, s_i$
$\text{slack}_i = \text{LST}_i - \text{EST}_i$
$$K_i = 1 - \frac{\text{slack}_i}{T^*}$$
$$\varepsilon_i = \varepsilon_{\max} - (\varepsilon_{\max} - \varepsilon_{\min})\, K_i$$

Zero slack → $K_i = 1$ → tightest budget

5. Placement Score · G3

$$S_{ij} = (1 + \eta K_i)\,\left(y_{ij}\, s_i + q_j + t_i / v_j\right) + \mu\, c_{ij}$$

$$j^* = \arg\min_{j \in J_i} S_{ij}$$

y_{ij} = 1 if j has no idle warm f_i. Critical → warm node; slack → cheapest node.

6. Overall Optimisation Objective

$$\min_{x,\,W}\ Z = \alpha\,\frac{T_{95}}{T_{\text{ref}}} + \beta\,P_{\text{cold}} + \gamma\,\frac{C}{C_{\text{ref}}} + \delta\,\frac{I}{I_{\text{ref}}}$$

assign $\sum_j x_{ij} = 1 \quad \forall f_i \in V$

memory $\sum_i m_i\,(x_{ij} + W_{ij}) \leq M_j \quad \forall j$

DAG order $\text{Start}_i \geq \text{Finish}_h \quad \forall (h,i) \in E$

SLO $T_{95} \leq T_{\text{SLO}}$

EST / LST: earliest / latest start time · Z terms normalised by the fixed keep-alive baseline

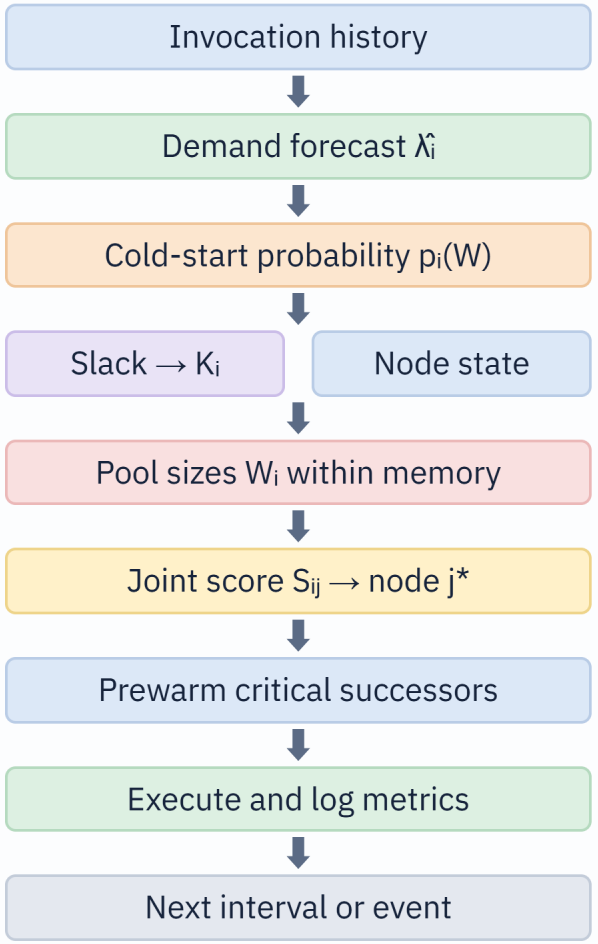
4 Pseudocode and Proposed Algorithm

Algorithm 1 · AP-CWS Scheduler

```
1  Input: DAGs G, traces λ, nodes J, ε_min, ε_max
2  every interval Δ do ▷ planning
3    for each f_i: λ̂_i ← ℓ_i + b_i ▷ Holt
4      ρ_i ← λ̂_i·t_i; d_i ← t_i + p_i·s_i
5    for each G: EST, LST by topological pass
6      K_i ← 1 - (LST_i - EST_i) / T*
7    for each f_i: ε_i ← ε_max - (ε_max-ε_min)·K_i
8      W_i ← min{ W : p_i(W) ≤ ε_i }
9    while Σ m_i W_i > free memory do
10     drop the container worth least per GB
11   prewarm / retire containers to reach W_i
12 on functions ready do ▷ dispatch
13   for f_i in ready queue by K_i (desc) do
14     J' ← { j : free memory on j ≥ m_i }
15     for j ∈ J': y_ij ← 0 if f_i warm on j, else 1
16       S_ij ← (1+ηK_i)(y_ij s_i + q_j + t_i/v_j) + μc_ij
17     j* ← argmin S_ij; run f_i on j*
18     for successor f_k with K_k ≥ κ do
19       prewarm f_k at time EST_k - s_k
20 record cold starts, latency, cost, idle memory
```

Cost per interval $O(|V| + |E| + |V| \cdot |J|)$ one DAG pass + one score per pair

Algorithm Flow



Baselines for Evaluation

1. Fixed 10-min keep-alive
2. Hybrid histogram (2020)
3. SSC, base paper (2024)
4. MPC prewarming (2025)
5. CASR caching (2025)
6. DROPS pools (2026)
7. **AP-CWS (proposed)**

Ablations

- $K_i = 0$: no workflow view
- Mean-sized pools
- No successor prewarm
- No cost term ($\mu = 0$)

Fair Comparison

Same memory for every policy; 5 seeds; mean ± 95% confidence interval.

κ: criticality threshold for just-in-time prewarming · lines 2–11 every minute, 12–19 on events

5 Simulator

faas-sim

Trace-driven serverless simulator

Raith et al., 2023 · Python + SimPy

Why faas-sim?

- Multi-node cluster + startup
- Pluggable scheduler, autoscaler
- Replays Azure Functions traces
- Open-source Python (SimPy)

Simulation Framework

Input

- Azure Functions 2019 trace
- Synthetic DAG shapes
- Node memory, speed, price
- AP-CWS and baseline policies

faas-sim + our layer

- SimPy discrete-event core
- Cluster, image pulls, startup
- DAG engine (added)
- Warm-pool controller (added)

Output

- Cold-start rate (all, critical path)
- p95 and mean workflow time
- Cost and idle warm memory
- SLO violations

Simulators Considered

Needs	SimFaaS	CloudSimSC	faas-sim
DAG workflows	No	No	Added by us
Several nodes	No	Yes	Yes
Pluggable scaling	No	Yes	Yes
Language	Python	Java	Python

SimFaaS models one function; we keep it to check $p_i(W)$.

Workload Types



Key Metrics

Cold-start rate · Workflow p95 · Idle memory · Resource cost · SLO violations

No public trace includes workflow structure, so DAG shapes are synthetic and arrival rates come from the trace

Thank you

Questions?

EXPECTED CONTRIBUTIONS

- Pool sizing as a chosen cold-start budget
- Criticality that counts expected cold starts
- AP-CWS: joint pool sizing and placement
- DAG layer for faas-sim; tests vs 2024–2026 work

NEXT STEPS

Phase 1 · now
Problem, model, algorithm

→

Phase 2
DAG layer, baselines

→

Phase 3
Experiments, paper

KEY REFERENCES

Pan et al., SSC, IEEE T-SUSC, 2024 (base paper)

Shahrad et al., Serverless in the Wild, ATC 2020

Hanaforoosh et al., granularity trees, FGCS 2025

Chen et al., CASR, FGCS 2025

Zhou et al., RL load balancing, JNCA 2025

Nguyen et al., MPC cold starts, MASCOTS 2025

Zolfaghari et al., edge workflows, IEEE TSC 2025

Alquraan et al., DROPS, EuroSys 2026

Raith et al., faas-sim, SPE 2023

Mahmoudi and Khazaei, SimFaaS, CLOSER 2021

Mampage et al., CloudSimSC, IEEE HPCC 2023

Nikhil Sharma · Raghav Arora · Jashanpreet Singh · Cloud Computing, IIT (BHU) Varanasi